Name : Shubham kumar
Add no: 25MT0517
IIT(ISM) dhanbad

Verilog coding
Day-04
Mux 4X1

Write a Verilog program to implement a **4×1 Multiplexer** using:

- Gate-Level Modeling
- Dataflow Modeling
- Behavioral Modeling
- 4×1 MUX using 2×1 MUX (Component Instantiation)
  Also, write a testbench to verify the design.

4X1 MUX USING DATAFLOW

```
module mux4x1_df(
   input i0, i1, i2, i3,
   input s1, s0,
   output y
);

  assign y = (~s1 & ~s0 & i0) |
             (~s1 &  s0 & i1) |
             ( s1 & ~s0 & i2) |
             ( s1 &  s0 & i3);

endmodule
```

Gate Level Modeling Mux4X1

```
module mux4x1_gl(
   input i0, i1, i2, i3,
   input s1, s0,
   output y
);

  wire s1n, s0n;
  wire w1, w2, w3, w4;

  not (s1n, s1);
  not (s0n, s0);
```

```
  and (w1, s1n, s0n, i0);
  and (w2, s1n, s0,  i1);
  and (w3, s1,  s0n, i2);
  and (w4, s1,  s0,  i3);

  or (y, w1, w2, w3, w4);

endmodule
```

Behavioral Modeling Mux4X1

```
module mux4x1_bh(
   input i0, i1, i2, i3,
   input s1, s0,
   output reg y
);

  always @(*) begin
   case ({s1, s0})
     2'b00: y = i0;
     2'b01: y = i1;
     2'b10: y = i2;
     2'b11: y = i3;
   endcase
  end
endmodule
```

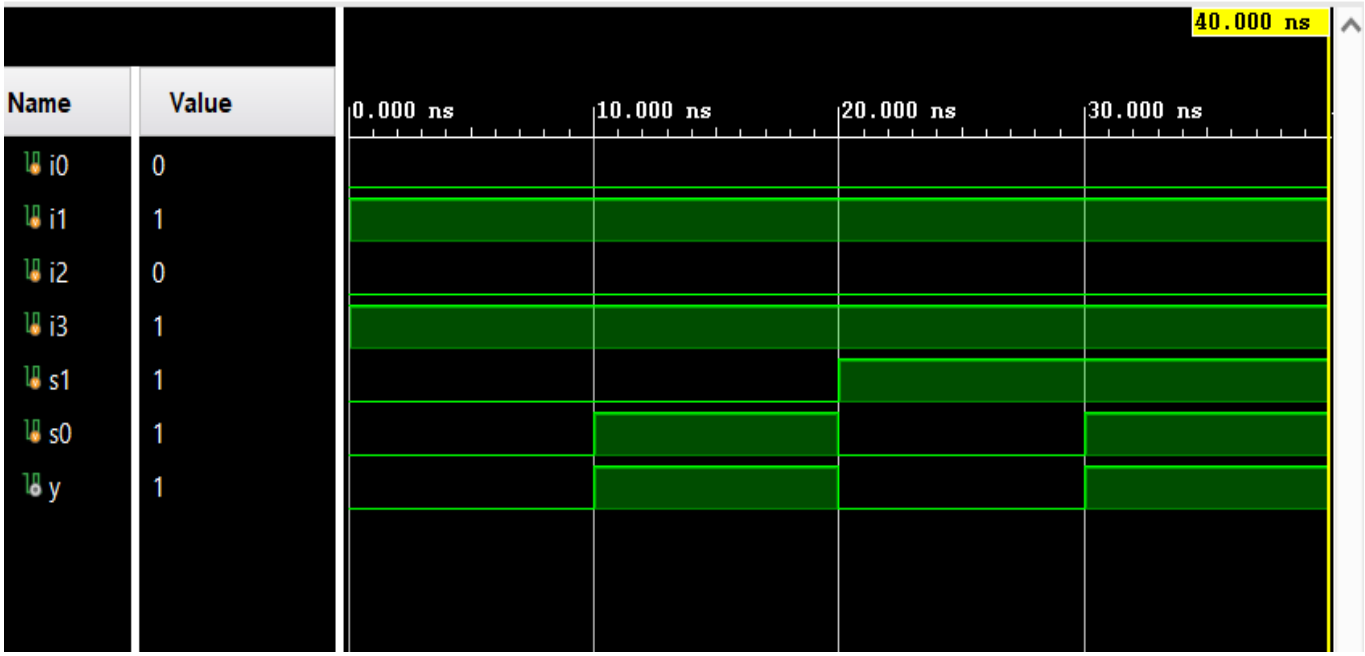
4×1 MUX using 2×1 MUX (Component Instantiation)

```
module mux4x1_st(
   input i0, i1, i2, i3,
   input s1, s0,
   output y
);

  wire w1, w2;

  mux2x1 M1 (i0, i1, s0, w1);
  mux2x1 M2 (i2, i3, s0, w2);

  mux2x1 M3 (w1, w2, s1, y);

endmodule
```

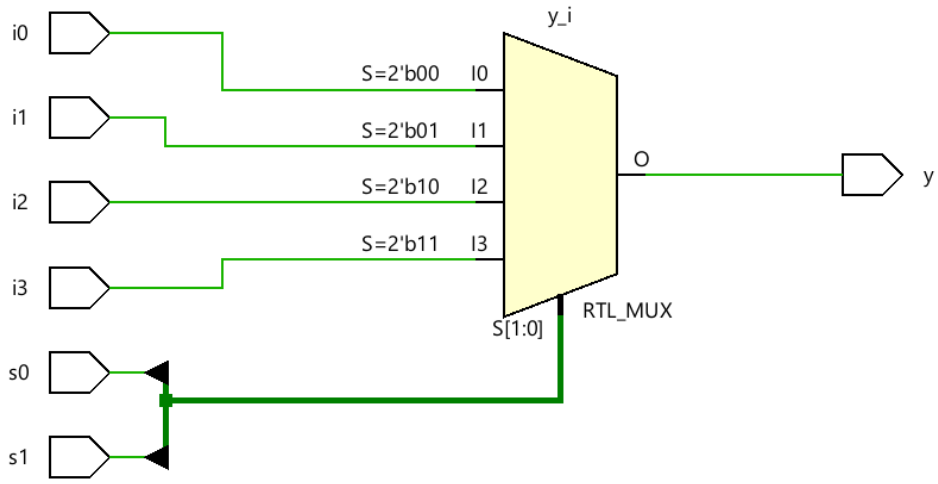
Test bench Mux 4X1

```
module tb_mux4x1;

  reg i0, i1, i2, i3;
  reg s1, s0;
  wire y;

  // Change module name if needed (df/gl/bh/st)
  mux4x1_bh DUT (.i0(i0), .i1(i1), .i2(i2), .i3(i3),
              .s1(s1), .s0(s0), .y(y));

  initial begin
    $monitor("%b  %b  |   %b", s1, s0, y);
    i0=0; i1=1; i2=0; i3=1;

    s1=0; s0=0; #10;
    s1=0; s0=1; #10;
    s1=1; s0=0; #10;
    s1=1; s0=1; #10;

    $finish;
  end

endmodule
```

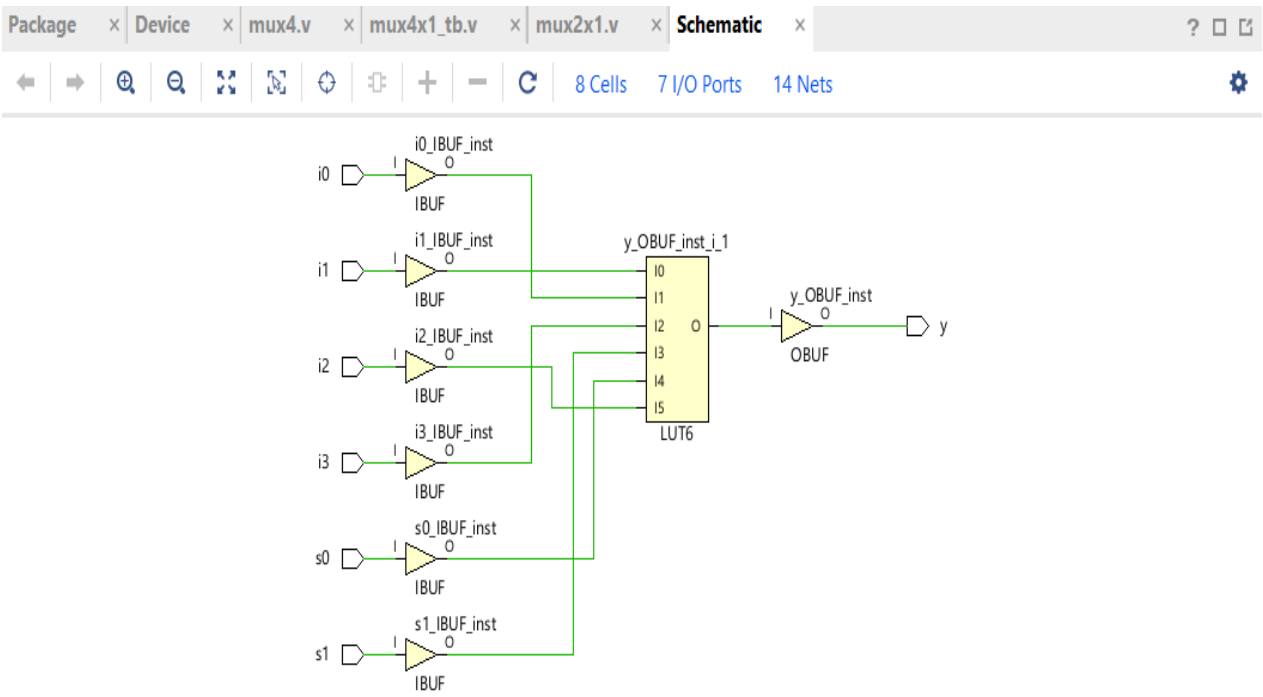
Mux4X1



RTL design Mux4X1



Schematic design Mux4X1



DEMUX 4X1

Write a Verilog program to implement a **1X4 DEMULTIPLEXER** using:

- Gate-Level Modeling
- Dataflow Modeling
- Behavioral Modeling
- 1×4 DEMUX using 1×2 DEMUX

  Also, write a testbench to verify the design.

1X2 DEMUX USING DATAFLOW

```
module demux1x4_df(
   input d,
   input s1, s0,
   output y0, y1, y2, y3
);

  assign y0 = d & ~s1 & ~s0;
  assign y1 = d & ~s1 &  s0;
  assign y2 = d &  s1 & ~s0;
  assign y3 = d &  s1 &  s0;

endmodule
```

Gate Level Modeling Demux 1X2

```
module demux1x4_gl(
   input d,
   input s1, s0,
   output y0, y1, y2, y3
);

  wire s1n, s0n;

  not (s1n, s1);
  not (s0n, s0);

  and (y0, d, s1n, s0n);
  and (y1, d, s1n, s0);
  and (y2, d, s1,  s0n);
  and (y3, d, s1,  s0);

endmodule
```

Behavioral Modeling Demux 1X2

```
 module demux1x4_bh(
   input d,
   input s1, s0,
   output reg y0, y1, y2, y3
);

  always @(*) begin
```

```
    // default all outputs = 0
    {y0, y1, y2, y3} = 4'b0000;

    case ({s1, s0})
      2'b00: y0 = d;
      2'b01: y1 = d;
      2'b10: y2 = d;
      2'b11: y3 = d;
    endcase
  end

endmodule
```

1×4 DEMUX using 1×2 DEMUX

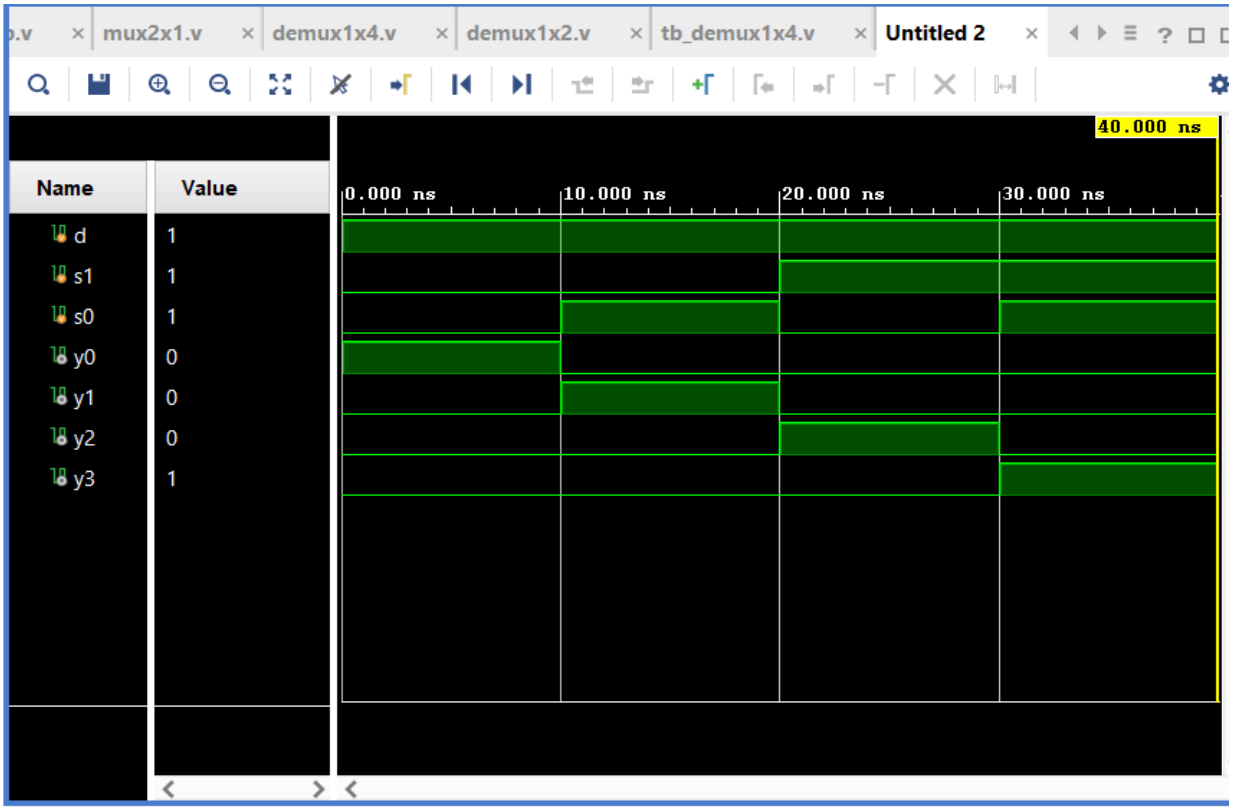
```
module demux1x4_st(
    input d,
    input s1, s0,
    output y0, y1, y2, y3
);

  wire w1, w2;

  demux1x2 D1 (d, s1, w1, w2);

  demux1x2 D2 (w1, s0, y0, y1);
  demux1x2 D3 (w2, s0, y2, y3);
endmodule
```

Test bench Demux 1X2

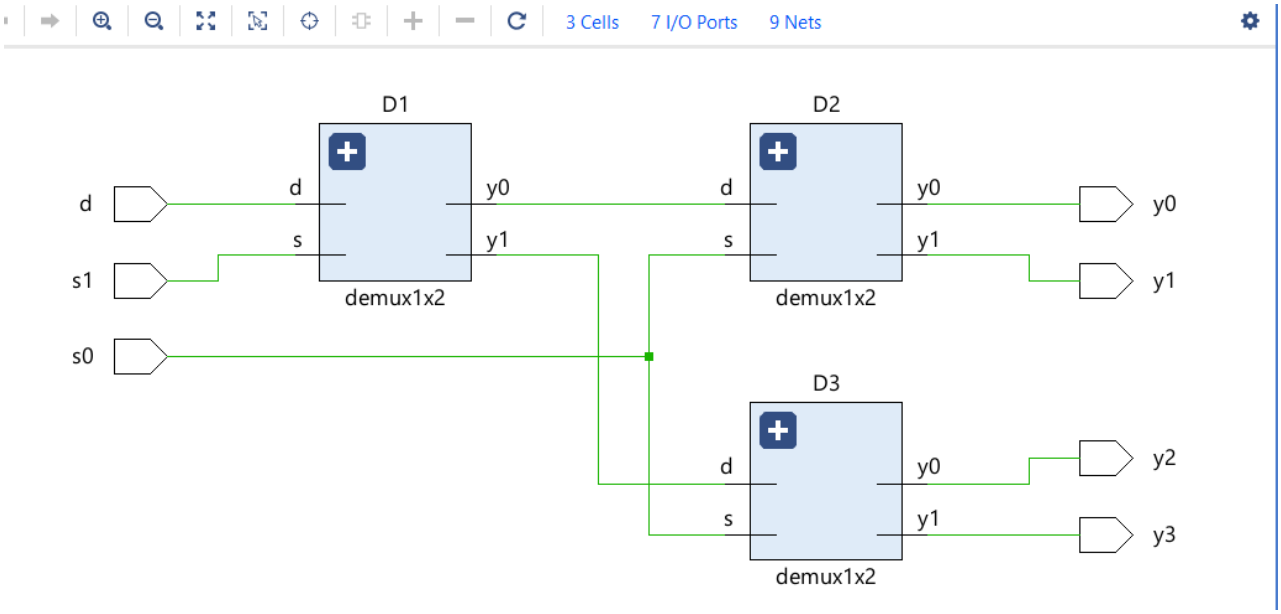
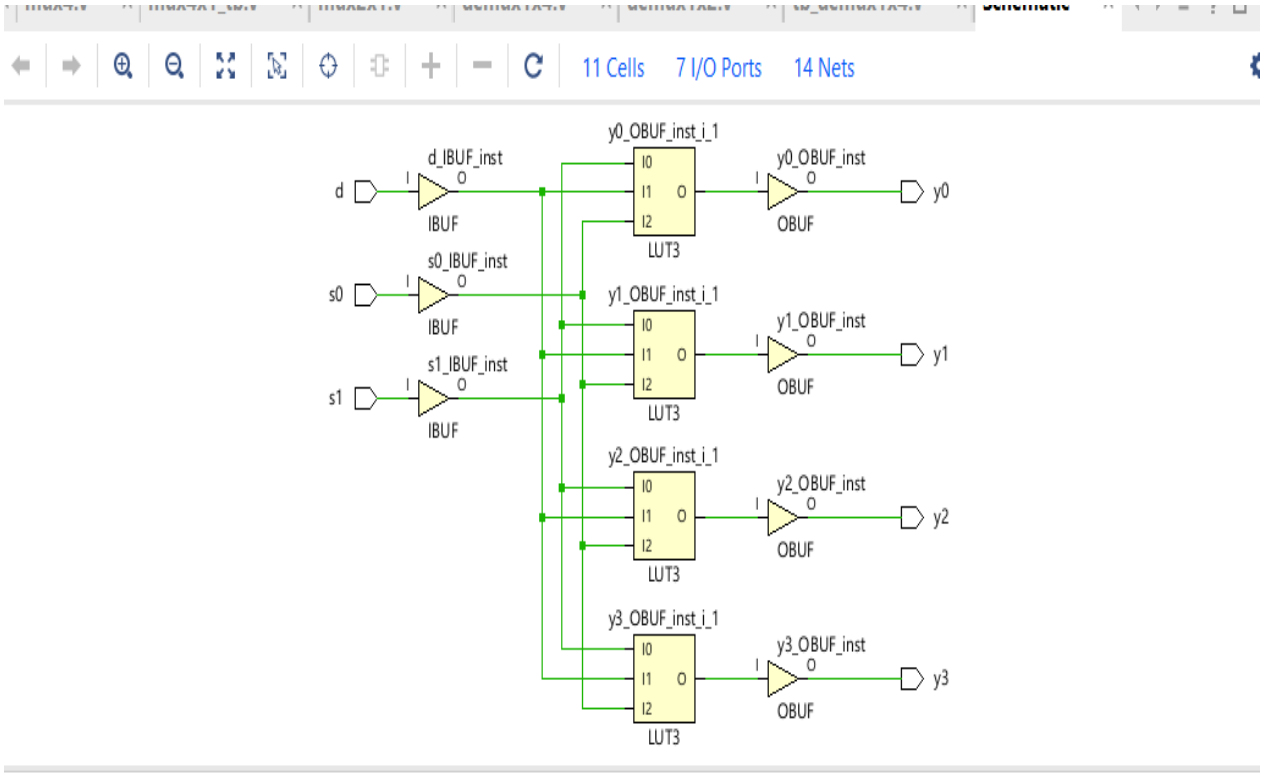
```
module tb_demux1x4;

  reg d;
  reg s1, s0;
  wire y0, y1, y2, y3;

  demux1x4_st DUT (.d(d), .s1(s1), .s0(s0),
                .y0(y0), .y1(y1), .y2(y2), .y3(y3));

  initial begin
    $display("S1 S0 | Y0 Y1 Y2 Y3");

    d = 1;
    s1=0; s0=0; #10;
```

```
  s1=0; s0=1; #10;
  s1=1; s0=0; #10;
  s1=1; s0=1; #10;
  $finish;
 end
endmodule
```

waveform Demux1x4



RTL design Demux1x4



Schematic design Demux1x4